

Python Programming

Day 4: 모듈, 예외 처리와 파일 입출력

Module, Exception Handling, File I/O

Contents

1	학습 목표	3
2	모듈과 import	3
2.1	모듈 전체 불러오기	3
2.2	특정 이름만 불러오기	4
2.3	와일드카드 import	4
2.4	최종 코드: 01_import.py	4
3	random 모듈	5
3.1	정수 난수: randint()	5
3.2	하나 선택하기: choice()	5
3.3	순서 섞기: shuffle()	5
3.4	여러 개 선택하기: sample()	6
3.5	조합과 sample()	6
3.6	함수 비교	6
3.7	최종 코드: 02_random.py	6
4	사용자 정의 모듈	7
4.1	함수 모듈 작성하기	7
4.2	모듈 불러오기	8
4.3	모듈 캐시: __pycache__	8
4.4	평균 함수 추가	8
5	예외 처리	9
5.1	기본 try--except	9
5.2	특정 예외 처리	9
5.3	예외 객체 확인하기	10
5.4	else	10
5.5	finally	11
5.6	직접 예외 발생시키기: raise	11
5.7	최종 코드: 04_try_except.py	11
6	파일 쓰기	12
6.1	파일 열기	12
6.2	write()와 줄바꿈	12
6.3	쓰기 모드의 덮어쓰기	12
6.4	이어쓰기 모드	13
6.5	최종 코드: 05_file_write.py	13
7	파일 읽기	13
7.1	파일 전체 읽기: read()	13
7.2	한 줄 읽기: readline()	14
7.3	모든 줄을 리스트로 읽기: readlines()	14
7.4	읽기 함수 비교	14
7.5	최종 코드: 06_file_read.py	14
8	안전한 파일 처리: with open()	15
8.1	기본 문법	15
8.2	쓰기 모드와 함께 사용하기	16
8.3	파일 객체를 직접 반복하기	16
8.4	최종 코드: 07_with_open.py	16

9	종합 실습: 학생 점수 관리 프로그램	17
9.1	입력과 유효성 검사	17
9.2	랜덤 점수와 평균	18
9.3	파일 저장과 형식 지정	18
9.4	저장한 결과 읽기	18
9.5	최종 코드: <code>practice_day04.py</code>	18
9.6	실행 예시	19
9.7	프로그램 구조 분석	19
10	실습 파일 목록	20
11	핵심 요약	20

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- `import`와 `from ... import ...`의 차이 설명하기
- `math` 모듈의 상수와 함수 사용하기
- `random` 모듈로 난수와 무작위 선택 구현하기
- Python 파일을 사용자 정의 모듈로 분리하여 불러오기
- `try`, `except`, `else`, `finally`로 예외 처리하기
- 필요한 경우 `raise`로 직접 예외 발생시키기
- 파일을 쓰기, 읽기, 이어쓰기 모드로 열기
- `read()`, `readline()`, `readlines()`의 차이 구분하기
- `with open()`으로 파일을 안전하게 처리하기
- 모듈, 예외 처리, 파일 입출력을 결합한 학생 점수 관리 프로그램 구현하기

2. 모듈과 `import`

프로그램이 커지면 모든 기능을 하나의 파일에 작성하기 어렵다. 모듈(module)은 관련된 변수, 함수, 클래스 등을 하나의 파일이나 라이브러리 단위로 묶은 것이다. 이미 만들어진 기능을 불러와 재사용하면 같은 코드를 반복해서 작성할 필요가 없다.

Python 표준 라이브러리는 Python 설치 시 함께 제공되는 모듈 모음이다. Day 4에서는 먼저 `math`와 `random`을 사용한다.

2.1. 모듈 전체 불러오기

`import math`는 `math` 모듈 전체를 불러온다. 모듈 안의 기능을 사용할 때는 모듈 이름과 점을 함께 작성한다.

```
1 import math
2
3 print(math.pi)
4 print(math.sqrt(25))
5 print(math.factorial(5))
```

출력 예시는 다음과 같다.

```
3.141592653589793
5.0
120
```

`math.pi`는 원주율을 저장한 상수이다. `math.sqrt(25)`는 25의 제곱근을 계산하며 결과를 실수 5.0으로 반환한다. `math.factorial(5)`는 5!을 계산한다.

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

핵심 포인트

`import math`를 사용하면 함수가 어느 모듈에서 왔는지 코드에 명확히 드러난다. `sqrt()`가 아니라 `math.sqrt()`라고 작성한다.

2.2. 특정 이름만 불러오기

모듈 전체가 아니라 필요한 이름만 직접 불러올 수도 있다.

```
1 from math import sqrt, pi, factorial
2
3 print(pi)
4 print(sqrt(25))
5 print(factorial(7))
```

이 방식에서는 모듈 이름을 앞에 붙이지 않는다.

방식	호출 형태
<code>import math</code>	<code>math.sqrt(25)</code>
<code>from math import sqrt</code>	<code>sqrt(25)</code>

2.3. 와일드카드 import

다음 문법은 모듈에서 공개된 여러 이름을 한꺼번에 불러온다.

```
1 from math import *
```

작성은 간단하지만 이름의 출처를 파악하기 어렵고, 현재 파일의 이름과 충돌할 수 있다. 따라서 일반적으로는 모듈 전체를 불러오거나 필요한 이름만 명시하는 방식이 더 안전하다.

2.4. 최종 코드: 01_import.py

```
1 import math
2
3 print("=== import math ===")
4 print(math.pi)
5 print(math.sqrt(25))
6 print(math.factorial(5))
7
8 print()
9
10 from math import sqrt, pi, factorial
11
12 print("=== from math import ... ===")
13 print(pi)
14 print(sqrt(25))
15 print(factorial(7))
```

실습

01_import.py를 실행하고 `sqrt()`, `factorial()`의 반환값과 자료형을 확인한다. 이후 `math` 모듈 전체 import 방식과 특정 이름 import 방식의 호출 형태를 비교한다.

3. random 모듈

random 모듈은 난수 생성과 무작위 선택에 사용된다. 게임, 시뮬레이션, 표본 추출, 테스트 데이터 생성 등에서 자주 활용된다.

컴퓨터의 일반적인 난수 생성기는 완전히 예측 불가능한 물리적 난수가 아니라 초기값과 알고리즘으로 생성되는 의사 난수(pseudorandom number)를 사용한다. 일반적인 학습과 시뮬레이션에서는 충분하지만, 암호나 보안 토큰에는 보안 목적의 별도 모듈을 사용해야 한다.

3.1. 정수 난수: randint()

```
1 import random
2
3 number = random.randint(1, 10)
4 print(number)
```

random.randint(a, b)는 a 이상 b 이하의 정수를 반환한다. 양 끝값이 모두 포함된다는 점이 중요하다.

$$P(X = k) = \frac{1}{b - a + 1}, \quad k \in \{a, a + 1, \dots, b\}$$

실행할 때마다 결과가 달라질 수 있다.

3.2. 하나 선택하기: choice()

```
1 import random
2
3 fruits = ["apple", "banana", "orange", "grape"]
4 print(random.choice(fruits))
5
6 word = "PYTHON"
7 print(random.choice(word))
```

choice()는 리스트나 문자열처럼 순서가 있는 객체에서 원소 하나를 무작위로 선택한다.

3.3. 순서 섞기: shuffle()

```
1 import random
2
3 cards = ["A", "K", "Q", "J", "10"]
4 print("Before:", cards)
5
6 random.shuffle(cards)
7
8 print("After :", cards)
```

shuffle()은 원본 리스트의 순서를 직접 바꾼다. 새로운 리스트를 반환하지 않으며 반환값은 None이다.

```
1 cards = ["A", "K", "Q", "J", "10"]
2 result = random.shuffle(cards)
3
4 print(cards)
5 print(result)
```

따라서 섞인 리스트가 필요하면 원래 리스트 변수인 cards를 사용해야 한다.

3.4. 여러 개 선택하기: sample()

```

1 import random
2
3 numbers = [1, 2, 3, 4, 5]
4 result = random.sample(numbers, 3)
5
6 print(result)
7 print(numbers)

```

sample(sequence, k)는 원본에서 서로 다른 원소 k 개를 선택하여 새로운 리스트로 반환한다. 원본은 바뀌지 않으며, 같은 원소를 중복 선택하지 않는다.

예를 들어 로또 번호 6개를 생성할 수 있다.

```

1 lotto = random.sample(range(1, 46), 6)
2 print(lotto)

```

3.5. 조합과 sample()

n 개 중 r 개를 순서 없이 선택하는 서로 다른 조합의 수는 다음과 같다.

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

sample()을 반복 호출하고 결과를 정렬한 튜플로 바꾸어 집합에 저장하면 여러 조합을 모을 수 있다.

```

1 import math
2 import random
3
4 items = ["A", "B", "C", "D"]
5 results = set()
6 target = math.comb(len(items), 2)
7
8 while len(results) < target:
9     selected = random.sample(items, 2)
10    results.add(tuple(sorted(selected)))
11
12 print(results)

```

이 방식은 가능한 조합을 언젠가 모두 발견할 수 있지만, 마지막 남은 조합이 언제 나올지 보장할 수 없다. 모든 조합을 체계적으로 생성할 때는 itertools.combinations()가 더 적합하다.

3.6. 함수 비교

함수	기능	원본 변경
randint(a, b)	정수 하나 생성	해당 없음
choice(seq)	원소 하나 선택	아니오
shuffle(list)	리스트 순서 섞기	예
sample(seq, k)	중복 없이 k 개 선택	아니오

3.7. 최종 코드: 02_random.py

```

1 import random
2
3 print("=== randint() ===")
4 number = random.randint(1, 10)

```

```

5 print(number)
6
7 print()
8
9 print("=== choice() ===")
10 fruits = ["apple", "banana", "orange", "grape"]
11 print(random.choice(fruits))
12
13 word = "PYTHON"
14 print(random.choice(word))
15
16 print()
17
18 print("=== shuffle() ===")
19 cards = ["A", "K", "Q", "J", "10"]
20 print("Before:", cards)
21
22 random.shuffle(cards)
23
24 print("After :", cards)
25
26 print()
27
28 print("=== sample() ===")
29 numbers = [1, 2, 3, 4, 5]
30 print(random.sample(numbers, 3))
31
32 students = ["Kim", "Lee", "Park", "Choi", "Jung", "Han"]
33 print(random.sample(students, 2))

```

실습

02_random.py에서 1부터 100까지의 정수 하나를 생성하고, 음식 목록에서 하나를 선택하며, 카드 목록을 섞고, 학생 목록에서 두 명을 중복 없이 선택한다.

4. 사용자 정의 모듈

Python 파일 하나는 다른 Python 파일에서 모듈로 불러올 수 있다. 계산 기능과 프로그램 실행 흐름을 파일별로 나누면 각 파일의 책임이 분명해지고 재사용도 쉬워진다.

이번 예제는 다음 두 파일로 구성된다.

03_custom_module.py

my_math.py

두 파일은 같은 day04 폴더에 둔다.

4.1. 함수 모듈 작성하기

my_math.py

```

1 def add(a, b):
2     return a + b
3
4
5 def subtract(a, b):
6     return a - b
7

```



```

8
9 def multiply(a, b):
10     return a * b

```

이 파일은 함수를 정의하지만 직접 실행 흐름을 담당하지 않는다.

4.2. 모듈 불러오기

03_custom_module.py

```

1 import my_math
2
3 print("=== Custom Module ===")
4
5 print(my_math.add(10, 20))
6 print(my_math.subtract(20, 7))
7 print(my_math.multiply(6, 8))

```

Python은 현재 파일과 같은 폴더에서 `my_math.py`를 찾고, 모듈 객체를 만든 뒤 `my_math`라는 이름에 연결한다. 이후 점 표기법으로 모듈 안의 함수를 호출한다.

핵심 포인트

모듈 파일 이름이 `my_math.py`이면 import할 때 확장자를 제외하고 `import my_math`라고 작성한다.

4.3. 모듈 캐시: `--pycache--`

사용자 정의 모듈을 import하면 Python이 같은 폴더에 `--pycache--` 폴더를 만들 수 있다. 이 폴더에는 다음과 같은 바이트코드 파일이 저장된다.

```

--pycache__/
    my_math.cpython-313.pyc

```

Python 소스는 실행 전에 바이트코드로 컴파일된다. 캐시가 있으면 다음 import에서 이미 만들어진 바이트코드를 재사용할 수 있다. 삭제해도 프로그램에는 문제가 없으며, 필요하면 Python이 다시 생성한다.

Git 저장소에는 일반적으로 캐시 파일을 포함하지 않는다.

```

--pycache__/
*.pyc

```

위 항목을 프로젝트 루트의 `.gitignore`에 추가할 수 있다.

4.4. 평균 함수 추가

종합 실습에서 사용할 평균 함수를 `my_math.py`에 추가한다.

```

1 def average(numbers):
2     return sum(numbers) / len(numbers)

```

최종 `my_math.py`는 다음과 같다.

```

1 def add(a, b):
2     return a + b
3
4
5 def subtract(a, b):
6     return a - b

```

```

7
8
9 def multiply(a, b):
10     return a * b
11
12
13 def average(numbers):
14     return sum(numbers) / len(numbers)

```

실습

my_math.py에 나눗셈 함수와 평균 함수를 추가한다. 03_custom_module.py에서 두 함수를 호출하여 결과를 확인한다.

5. 예외 처리

프로그램 실행 중 발생하는 오류 상황을 예외(exception)라고 한다. 예외를 처리하지 않으면 프로그램은 오류 메시지를 출력하고 즉시 종료된다.

예를 들어 int()는 정수로 변환할 수 없는 문자열을 받으면 ValueError를 발생시킨다.

```

1 number = int(input("Enter a number: "))
2 print(number)

```

사용자가 hello를 입력하면 변환이 불가능하므로 프로그램이 중단된다.

5.1. 기본 try--except

```

1 try:
2     number = int(input("Enter a number: "))
3     print(number)
4
5 except:
6     print("Invalid input!")

```

Python은 먼저 try 블록을 실행한다. 예외가 발생하면 남은 try 코드를 건너뛰고 except 블록을 실행한다.

핵심 포인트

except:는 모든 종류의 예외를 잡는다. 간단한 실험에는 편리하지만, 예상하지 못한 문제까지 숨길 수 있으므로 구체적인 예외 종류를 지정하는 편이 좋다.

5.2. 특정 예외 처리

```

1 try:
2     number = int(input("Enter a number: "))
3     print(number)
4
5 except ValueError:
6     print("Please enter an integer.")

```

나눗셈 프로그램에는 두 종류의 예외가 발생할 수 있다.

```

1 try:
2     x = int(input("x: "))
3     y = int(input("y: "))

```

```

4     print(x / y)
5
6
7 except ValueError:
8     print("Please enter integers.")
9
10 except ZeroDivisionError:
11     print("Cannot divide by zero.")

```

ValueError는 입력값을 정수로 변환할 수 없을 때 발생하고, ZeroDivisionError는 0으로 나눌 때 발생한다.

5.3. 예외 객체 확인하기

```

1 try:
2     print(10 / 0)
3
4 except Exception as e:
5     print(type(e))
6     print(e)

```

Exception as e는 발생한 예외 객체를 e라는 이름에 저장한다. 개발 중에는 예외의 종류와 메시지를 확인하는 데 유용하다.

자주 만나는 예외는 다음과 같다.

예외	대표 상황
ValueError	값의 형식이 올바르지 않음
ZeroDivisionError	0으로 나눔
TypeError	허용되지 않는 자료형 조합
IndexError	리스트 범위를 벗어난 인덱스
KeyError	딕셔너리에 없는 키 조회
FileNotFoundError	존재하지 않는 파일 열기

모든 예외 이름을 미리 외울 필요는 없다. 프로그램을 실행하고 오류 메시지에서 예외 종류를 확인한 뒤 적절한 except를 추가하는 방식으로 학습할 수 있다.

5.4. else

else 블록은 try 안에서 예외가 발생하지 않았을 때만 실행된다.

```

1 try:
2     number = int(input("Enter a number: "))
3     result = 100 / number
4
5 except ValueError:
6     print("Please enter an integer.")
7
8 except ZeroDivisionError:
9     print("Zero is not allowed.")
10
11 else:
12     print("Result:", result)

```

예외가 발생할 가능성이 있는 코드와, 성공했을 때 수행할 코드를 분리할 수 있다.

5.5. finally

finally 블록은 예외 발생 여부와 관계없이 항상 실행된다.

```

1 try:
2     number = int(input("Enter a number: "))
3     print(100 / number)
4
5 except ValueError:
6     print("Please enter an integer.")
7
8 except ZeroDivisionError:
9     print("Zero is not allowed.")
10
11 finally:
12     print("Program finished.")

```

파일 닫기, 네트워크 연결 종료, 임시 자원 정리처럼 반드시 수행해야 하는 작업을 배치할 수 있다.

5.6. 직접 예외 발생시키기: raise

어떤 값이 Python 문법상 올바르더라도 프로그램 규칙에 맞지 않을 수 있다. 과목 수는 정수여야 할 뿐 아니라 양수여야 한다.

```

1 subject_count = int(input("Number of subjects: "))
2
3 if subject_count <= 0:
4     raise ValueError

```

0과 음수는 int() 변환에는 성공하지만 과목 수로는 부적절하다. raise ValueError를 이용하면 개발자가 직접 예외를 발생시킬 수 있다.

5.7. 최종 코드: 04_try_except.py

```

1 try:
2     number = int(input("Enter a number: "))
3     result = 100 / number
4
5 except ValueError:
6     print("Please enter an integer.")
7
8 except ZeroDivisionError:
9     print("Zero is not allowed.")
10
11 except Exception as e:
12     print("Unexpected error:", e)
13
14 else:
15     print("Result:", result)
16
17 finally:
18     print("Program finished.")

```

실습

04_try_except.py에 25, 0, hello를 각각 입력한다. 어떤 블록이 실행되는지 확인하고, 모든 경우에 finally가 실행되는지 관찰한다.

6. 파일 쓰기

프로그램의 변수는 일반적으로 프로그램이 종료되면 사라진다. 실행이 끝난 뒤에도 데이터를 보존하려면 파일에 기록해야 한다. 게임 저장, 설정 파일, 사용자 정보, 실험 데이터와 로그가 대표적인 예이다.

6.1. 파일 열기

```
1 file = open("hello.txt", "w")
2
3 file.write("Hello Python!")
4
5 file.close()
```

`open()`은 파일 객체를 반환한다. 첫 번째 인자는 파일 이름, 두 번째 인자는 파일을 여는 모드이다.

모드	의미
"r"	읽기
"w"	쓰기 및 덮어쓰기
"a"	기존 내용 뒤에 이어쓰기

6.2. `write()`와 줄바꿈

`write()`는 문자열을 파일에 기록한다. 자동으로 줄바꿈하지 않기 때문에 줄을 나누려면 `\n`을 직접 작성해야 한다.

```
1 file = open("student.txt", "w")
2
3 file.write("Kim\n")
4 file.write("Lee\n")
5 file.write("Park\n")
6
7 file.close()
```

파일 내용은 다음과 같다.

```
Kim
Lee
Park
```

6.3. 쓰기 모드의 덮어쓰기

"w" 모드로 기존 파일을 열면 이전 내용이 지워진다.

```
1 file = open("hello.txt", "w")
2 file.write("First")
3 file.close()
4
5 file = open("hello.txt", "w")
6 file.write("Second")
7 file.close()
```

최종 파일에는 Second만 남는다.

핵심 포인트

"w"는 파일이 없으면 새로 만들고, 파일이 이미 있으면 기존 내용을 지운 뒤 다시 쓴다. 기존 데이터를 보존해야 할 때는 주의해야 한다.

6.4. 이어쓰기 모드

기존 내용 뒤에 추가하려면 "a" 모드를 사용한다.

```
1 file = open("log.txt", "a")
2 file.write("Program started.\n")
3 file.close()
```

프로그램을 여러 번 실행하면 각 기록이 파일 끝에 계속 추가된다.

6.5. 최종 코드: 05_file_write.py

```
1 file = open("hello.txt", "w")
2
3 file.write("Hello Python!")
4
5 file.close()
6
7 file = open("student.txt", "w")
8
9 file.write("Kim\n")
10 file.write("Lee\n")
11 file.write("Park\n")
12
13 file.close()
14
15 file = open("message.txt", "w")
16
17 file.write("Python\n")
18 file.write("is\n")
19 file.write("awesome!")
20
21 file.close()
```

실습

05_file_write.py를 실행한 뒤 hello.txt, student.txt, message.txt의 내용을 직접 확인한다. 같은 파일을 "w"로 다시 열어 기존 내용이 사라지는 것도 확인한다.

7. 파일 읽기

파일을 읽을 때는 "r" 모드를 사용한다. 읽기 모드는 파일이 존재하지 않으면 FileNotFoundError를 발생시킨다.

7.1. 파일 전체 읽기: read()

```
1 file = open("message.txt", "r")
2
3 content = file.read()
4
5 print(content)
6
7 file.close()
```

read()는 파일 전체 내용을 하나의 문자열로 반환한다.

Python\nis\nawesome!

작은 파일을 한 번에 처리할 때 편리하지만, 매우 큰 파일에서는 전체 내용을 메모리에 올리므로 부담이 될 수 있다.

7.2. 한 줄 읽기: readline()

```
1 file = open("message.txt", "r")
2
3 print(file.readline().strip())
4 print(file.readline().strip())
5 print(file.readline().strip())
6
7 file.close()
```

readline()은 현재 위치에서 한 줄을 읽는다. 줄 끝의 \n도 포함될 수 있으므로 출력할 때 strip()으로 제거할 수 있다.

파일 객체는 읽은 위치를 기억한다. 첫 번째 호출은 첫 줄, 두 번째 호출은 두 번째 줄을 반환한다.

7.3. 모든 줄을 리스트로 읽기: readlines()

```
1 file = open("message.txt", "r")
2
3 lines = file.readlines()
4 print(lines)
5
6 file.close()
```

출력은 다음과 같다.

```
['Python\n', 'is\n', 'awesome!']
```

각 줄이 리스트의 원소가 되므로 반복문으로 처리할 수 있다.

```
1 file = open("student.txt", "r")
2
3 lines = file.readlines()
4
5 for line in lines:
6     print(line.strip())
7
8 file.close()
```

7.4. 읽기 함수 비교

함수	반환 형태	특징
read()	문자열 하나	파일 전체를 한 번에 읽음
readline()	문자열 하나	현재 위치에서 한 줄 읽음
readlines()	문자열 리스트	모든 줄을 리스트로 읽음

7.5. 최종 코드: 06_file_read.py

```
1 file = open("message.txt", "r")
2
3 content = file.read()
4
5 print("=== read() ===")
6 print(content)
```

```

7
8 file.close()
9
10 print()
11
12 file = open("message.txt", "r")
13
14 print("=== readline() ===")
15 print(file.readline().strip())
16 print(file.readline().strip())
17 print(file.readline().strip())
18
19 file.close()
20
21 print()
22
23 file = open("message.txt", "r")
24
25 lines = file.readlines()
26
27 print("=== readlines() ===")
28 print(lines)
29
30 file.close()
31
32 print()
33
34 file = open("student.txt", "r")
35
36 lines = file.readlines()
37
38 print("=== Student List ===")
39 for line in lines:
40     print(line.strip())
41
42 file.close()

```

실습

06_file_read.py에서 동일한 파일을 read(), readline(), readlines()로 각각 읽는다. 반환 값의 자료형과 구조를 비교한다.

8. 안전한 파일 처리: with open()

직접 open()과 close()를 사용하면 파일을 닫는 코드를 잊거나, 닫기 전에 예외가 발생할 수 있다.

```

1 file = open("message.txt", "r")
2
3 print(10 / 0)
4
5 file.close()

```

나눗셈에서 예외가 발생하면 file.close()에 도달하지 못한다. 이를 해결하기 위해 Python에서는 컨텍스트 관리자 문법인 with를 사용한다.

8.1. 기본 문법


```

1 with open("message.txt", "r") as file:
2     content = file.read()
3
4 print(content)

```

with 블록을 벗어나면 파일이 자동으로 닫힌다. 예외가 발생하더라도 정리 과정이 수행된다. 개념적으로는 다음 구조와 비슷하다.

```

1 file = open("message.txt", "r")
2
3 try:
4     content = file.read()
5
6 finally:
7     file.close()

```

8.2. 쓰기 모드와 함께 사용하기

```

1 with open("practice.txt", "w") as file:
2     file.write("Python\n")
3     file.write("File IO\n")
4     file.write("with open")

```

블록이 끝나면 close()를 직접 호출하지 않아도 파일이 닫힌다.

8.3. 파일 객체를 직접 반복하기

파일 객체는 반복 가능한 객체이므로 한 줄씩 직접 순회할 수 있다.

```

1 with open("student.txt", "r") as file:
2     for line in file:
3         print(line.strip())

```

이 방식은 파일 전체를 리스트에 저장하지 않는다. 한 줄을 읽고 처리한 뒤 다음 줄로 이동하므로 큰 파일에서도 메모리를 효율적으로 사용할 수 있다.

방식	메모리 사용
file.readlines()	모든 줄을 리스트로 저장
for line in file	한 줄씩 순차 처리

핵심 포인트

일반적인 파일 처리에서는 with open()을 기본 방식으로 사용하는 것이 좋다. 자동으로 파일을 닫아 주므로 코드가 짧고 안전하다.

8.4. 최종 코드: 07_with_open.py

```

1 print("=== Read Entire File ===")
2
3 with open("message.txt", "r") as file:
4     content = file.read()
5
6 print(content)
7
8 print()
9

```

```

10 print("=== Write File ===")
11
12 with open("practice.txt", "w") as file:
13     file.write("Python\n")
14     file.write("File IO\n")
15     file.write("with open")
16
17 print("practice.txt was created.")
18
19 print()
20
21 print("=== Read Line by Line ===")
22
23 with open("student.txt", "r") as file:
24     for line in file:
25         print(line.strip())

```

실습

07_with_open.py에서 파일을 쓰고 다시 읽는다. 모든 file.close()를 제거한 상태에서도 파일이 정상적으로 저장되는지 확인한다.

9. 종합 실습: 학생 점수 관리 프로그램

Day 4의 종합 실습은 모듈, 난수, 예외 처리와 파일 입출력을 하나의 프로그램으로 결합한다. 프로그램의 흐름은 다음과 같다.

1. 학생 이름을 입력받는다.
2. 과목 수를 정수로 입력받는다.
3. 잘못된 과목 수를 예외로 처리한다.
4. 과목 수만큼 무작위 점수를 생성한다.
5. 사용자 정의 모듈의 함수로 평균을 계산한다.
6. 결과를 scores.txt에 저장한다.
7. 저장한 파일을 다시 읽어 화면에 출력한다.

9.1. 입력과 유효성 검사

```

1 name = input("Student name: ")
2
3 try:
4     subject_count = int(input("Number of subjects: "))
5
6     if subject_count <= 0:
7         raise ValueError
8
9 except ValueError:
10    print("Please enter a positive integer.")

```

int() 변환에 실패한 경우와 0 이하를 입력한 경우를 모두 ValueError로 처리한다.

9.2. 랜덤 점수와 평균

```

1 scores = []
2
3 for _ in range(subject_count):
4     scores.append(random.randint(60, 100))
5
6 average = my_math.average(scores)

```

반복 변수의 값이 필요하지 않으므로 관례적으로 `_`를 사용한다. 점수는 60점 이상 100점 이하에서 생성된다.

평균은 사용자 정의 모듈의 `average()` 함수가 계산한다.

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

9.3. 파일 저장과 형식 지정

```

1 with open("scores.txt", "w") as file:
2     file.write(f"Student: {name}\n")
3     file.write(f"Scores: {scores}\n")
4     file.write(f"Average: {average:.2f}\n")

```

f-string은 문자열 안에 변수 값을 삽입한다. `{average:.2f}`는 평균을 소수점 아래 둘째 자리까지 표시한다.

9.4. 저장한 결과 읽기

```

1 print("\n==== Saved Data =====")
2
3 with open("scores.txt", "r") as file:
4     for line in file:
5         print(line.strip())

```

파일 객체를 반복하여 한 줄씩 출력한다.

9.5. 최종 코드: practice_day04.py

```

1 import random
2 import my_math
3
4 name = input("Student name: ")
5
6 try:
7     subject_count = int(input("Number of subjects: "))
8
9     if subject_count <= 0:
10         raise ValueError
11
12 except ValueError:
13     print("Please enter a positive integer.")
14
15 else:
16     scores = []
17
18     for _ in range(subject_count):
19         scores.append(random.randint(60, 100))

```

```

20
21     average = my_math.average(scores)
22
23     print("Scores:", scores)
24     print(f"Average: {average:.2f}")
25
26     with open("scores.txt", "w") as file:
27         file.write(f"Student: {name}\n")
28         file.write(f"Scores: {scores}\n")
29         file.write(f"Average: {average:.2f}\n")
30
31     print("\n===== Saved Data =====")
32
33     with open("scores.txt", "r") as file:
34         for line in file:
35             print(line.strip())

```

9.6. 실행 예시

Student name: Kim
 Number of subjects: 3
 Scores: [82, 95, 76]
 Average: 84.33

===== Saved Data =====
 Student: Kim
 Scores: [82, 95, 76]
 Average: 84.33

잘못된 입력의 예시는 다음과 같다.

Student name: Kim
 Number of subjects: 0
 Please enter a positive integer.

또는

Student name: Kim
 Number of subjects: hello
 Please enter a positive integer.

9.7. 프로그램 구조 분석

기능	사용한 개념
점수 무작위 생성	<code>random.randint()</code>
평균 계산	사용자 정의 모듈
잘못된 입력 처리	<code>try--except, raise</code>
정상 흐름 분리	<code>else</code>
결과 저장	<code>with open(..., "w")</code>
결과 불러오기	<code>with open(..., "r")</code>
한 줄씩 처리	<code>for line in file</code>

핵심 포인트

이 프로그램은 계산 기능을 `my_math.py`에 분리하고, 실행 흐름을 `practice_day04.py`에서 관리한다. 기능 분리, 오류 처리, 데이터 보존을 한 번에 연습하는 예제이다.

10. 실습 파일 목록

실습

Day 4의 학습 파일은 다음과 같이 관리한다.

- 01_import.py
- 02_random.py
- 03_custom_module.py
- 04_try_except.py
- 05_file_write.py
- 06_file_read.py
- 07_with_open.py
- my_math.py
- practice_day04.py
- hello.txt
- message.txt
- scores.txt
- student.txt

11. 핵심 요약

- 모듈은 관련 기능을 파일이나 라이브러리 단위로 묶는다.
- `import module`은 `module.name` 형태로 사용한다.
- `from module import name`은 지정한 이름을 직접 사용한다.
- `randint()`는 범위의 양 끝을 포함한다.
- `choice()`는 하나를 선택하고, `sample()`은 중복 없이 여러 개를 선택한다.
- `shuffle()`은 원본 리스트를 직접 수정하며 `None`을 반환한다.
- 다른 Python 파일을 `import`하여 사용자 정의 모듈로 사용할 수 있다.
- `__pycache__`는 `import` 성능을 위한 바이트코드 캐시이다.
- `try`에서 예외가 발생하면 해당 `except`로 이동한다.
- `else`는 예외가 없을 때, `finally`는 항상 실행된다.
- `raise`는 프로그램 규칙에 따라 직접 예외를 발생시킨다.
- `"w"`는 파일을 덮어쓰고, `"r"`은 파일을 읽으며, `"a"`는 기존 내용 뒤에 추가한다.
- `read()`는 전체 문자열, `readline()`은 한 줄, `readlines()`는 줄 리스트를 반환한다.
- `with open()`은 블록이 끝날 때 파일을 자동으로 닫는다.

- 큰 파일은 `for line in file` 방식으로 한 줄씩 처리하는 것이 효율적이다.